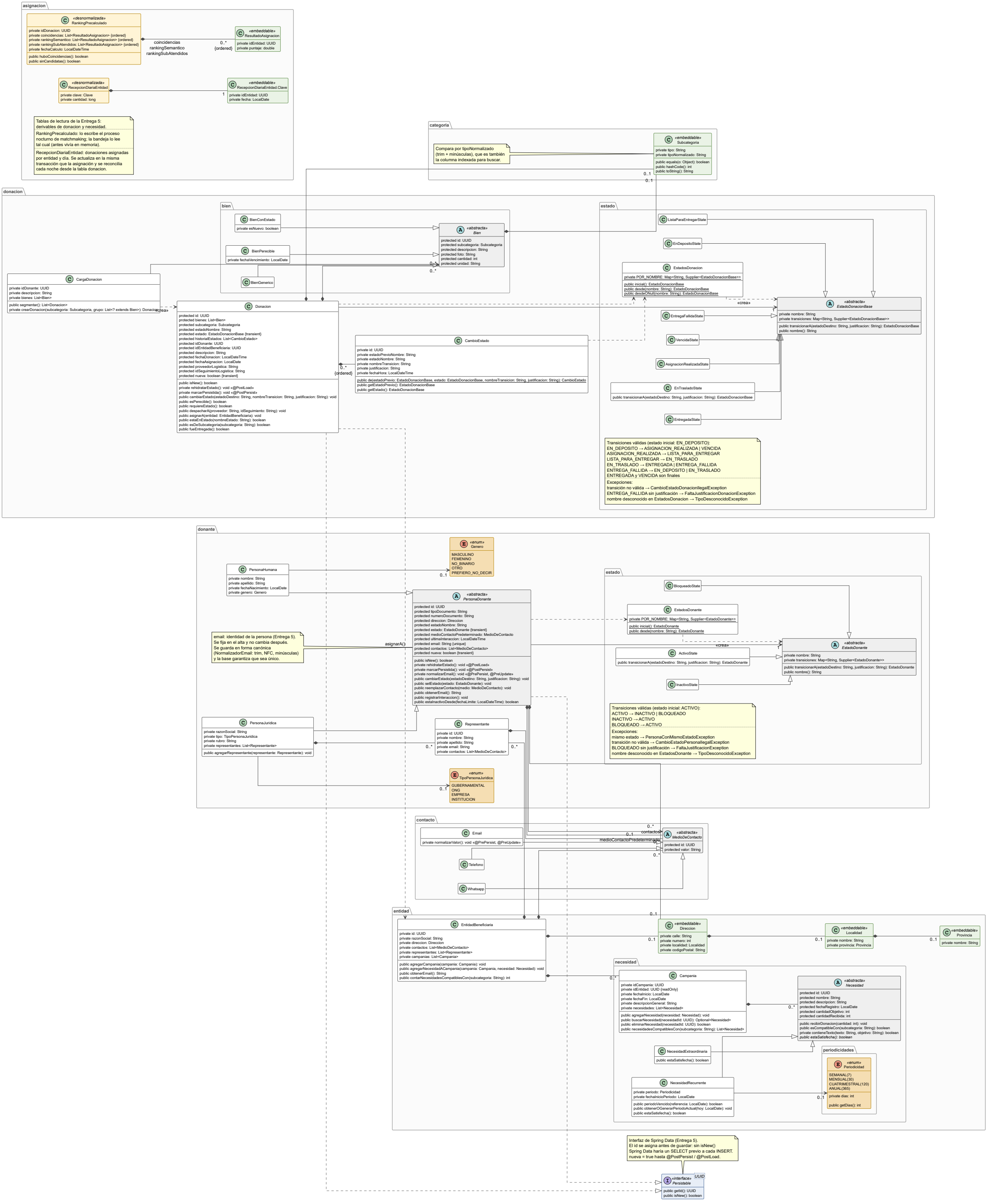

asignacion
«desnormalizada»
RankingPrecalculado
private idDonacion: UUID
private coincidencias: List<ResultadoAsignacion> {ordered}
private rankingSemantico: List<ResultadoAsignacion> {ordered}
private rankingSubAtendidos: List<ResultadoAsignacion> {ordered}
private fechaCalculo: LocalDateTime
public huboCoincidencias(): boolean
public sinCandidatas(): boolean
coincidencias
rankingSemantico
rankingSubAtendidos
0..*
{ordered}
«embeddable»
ResultadoAsignacion
private idEntidad: UUID
private puntaje: double
«desnormalizada»
RecepcionDiariaEntidad
private clave: Clave
private cantidad: long
1
«embeddable»
RecepcionDiariaEntidad.Clave
private idEntidad: UUID
private fecha: LocalDate
Tablas de lectura de la Entrega 5:
derivables de donacion y necesidad.
RankingPrecalculado: lo escribe el proceso nocturno de matchmaking; la bandeja lo lee tal cual (antes vivía en memoria).
RecepcionDiariaEntidad: donaciones asignadas por entidad y día. Se actualiza en la misma transacción que la asignación y se reconcilia cada noche desde la tabla donacion.
categoria
Compara por tipoNormalizado (trim + minúsculas), que es también la columna indexada para buscar.
«embeddable»
Subcategoria
private tipo: String
private tipoNormalizado: String
public equals(o: Object): boolean
public hashCode(): int
public toString(): String
0..1
0..1
donacion
bien
BienConEstado
private esNuevo: boolean
«abstracta»
Bien
protected id: UUID
protected subcategoria: Subcategoria
protected descripcion: String
protected foto: String
protected cantidad: int
protected unidad: String
BienPerecible
private fechaVencimiento: LocalDate
BienGenerico
0..*
0..*
CargaDonacion
private idDonante: UUID
private descripcion: String
private bienes: List<Bien>
public segmentar(): List<Donacion>
private crearDonacion(subcategoria: Subcategoria, grupo: List<? extends Bien>): Donacion
«crea»
Donacion
protected id: UUID
protected bienes: List<Bien>
protected subcategoria: Subcategoria
protected estadoNombre: String
protected estado: EstadoDonacionBase {transient}
protected historialEstados: List<CambioEstado>
protected idDonante: UUID
protected idEntidadBeneficiaria: UUID
protected descripcion: String
protected fechaDonacion: LocalDateTime
protected fechaAsignacion: LocalDate
protected proveedorLogistica: String
protected idSeguimientoLogistica: String
protected nueva: boolean {transient}
public isNew(): boolean
private rehidratarEstado(): void «@PostLoad»
private marcarPersistida(): void «@PostPersist»
public cambiarEstado(estadoDestino: String, nombreTransicion: String, justificacion: String): void
public esPerecible(): boolean
public requiereEstado(): boolean
public despacharA(proveedor: String, idSeguimiento: String): void
public asignarA(entidad: EntidadBeneficiaria): void
public estaEnEstado(nombreEstado: String): boolean
public esDeSubcategoria(subcategoria: String): boolean
public fueEntregada(): boolean
CambioEstado
private id: UUID
private estadoPrevioNombre: String
private estadoNombre: String
private nombreTransicion: String
private justificacion: String
private fechaHora: LocalDateTime
public de(estadoPrevio: EstadoDonacionBase, estado: EstadoDonacionBase, nombreTransicion: String, justificacion: String): CambioEstado
public getEstadoPrevio(): EstadoDonacionBase
public getEstado(): EstadoDonacionBase
0..*
{ordered}
estado
ListaParaEntregarState
EnDepositoState
EstadosDonacion
private POR_NOMBRE: Map<String, Supplier<EstadoDonacionBase>>
public inicial(): EstadoDonacionBase
public desde(nombre: String): EstadoDonacionBase
public desdeONull(nombre: String): EstadoDonacionBase
«crea»
1
«abstracta»
EstadoDonacionBase
private nombre: String
private transiciones: Map<String, Supplier<EstadoDonacionBase>>
public transicionarA(estadoDestino: String, justificacion: String): EstadoDonacionBase
public nombre(): String
EntregaFallidaState
VencidaState
AsignacionRealizadaState
EnTrasladoState
public transicionarA(estadoDestino: String, justificacion: String): EstadoDonacionBase
EntregadaState
Transiciones válidas (estado inicial: EN_DEPOSITO):
EN_DEPOSITO → ASIGNACION_REALIZADA | VENCIDA
ASIGNACION_REALIZADA → LISTA_PARA_ENTREGAR
LISTA_PARA_ENTREGAR → EN_TRASLADO
EN_TRASLADO → ENTREGADA | ENTREGA_FALLIDA
ENTREGA_FALLIDA → EN_DEPOSITO | EN_TRASLADO
ENTREGADA y VENCIDA son finales
Excepciones:
transición no válida → CambioEstadoDonacionIllegalException
ENTREGA_FALLIDA sin justificación → FaltaJustificacionDonacionException
nombre desconocido en EstadosDonacion → TipoDesconocidoException
donante
«enum»
Genero
MASCULINO
FEMENINO
NO_BINARIO
OTRO
PREFIERO_NO_DECIR
PersonaHumana
private nombre: String
private apellido: String
private fechaNacimiento: LocalDate
private genero: Genero
0..1
«abstracta»
PersonaDonante
protected id: UUID
protected tipoDocumento: String
protected numeroDocumento: String
protected direccion: Direccion
protected estadoNombre: String
protected estado: EstadoDonante {transient}
protected medioContactoPredeterminado: MedioDeContacto
protected ultimaInteraccion: LocalDateTime
protected email: String {unique}
protected contactos: List<MedioDeContacto>
protected nueva: boolean {transient}
public isNew(): boolean
private rehidratarEstado(): void «@PostLoad»
private marcarPersistida(): void «@PostPersist»
private normalizarEmail(): void «@PrePersist, @PreUpdate»
public cambiarEstado(estadoDestino: String, justificacion: String): void
public setEstado(estado: EstadoDonante): void
public reemplazarContacto(medio: MedioDeContacto): void
public obtenerEmail(): String
public registrarInteraccion(): void
public estaInactivoDesde(fechaLimite: LocalDateTime): boolean
email: identidad de la persona (Entrega 5).
Se fija en el alta y no cambia después.
Se guarda en forma canónica
(NormalizadorEmail: trim, NFC, minúsculas)
y la base garantiza que sea único.
asignarA()
estado
BloqueadoState
EstadosDonante
private POR_NOMBRE: Map<String, Supplier<EstadoDonante>>
public inicial(): EstadoDonante
public desde(nombre: String): EstadoDonante
«crea»
1
«abstracta»
EstadoDonante
private nombre: String
private transiciones: Map<String, Supplier<EstadoDonante>>
public transicionarA(estadoDestino: String, justificacion: String): EstadoDonante
public nombre(): String
ActivoState
public transicionarA(estadoDestino: String, justificacion: String): EstadoDonante
InactivoState
Transiciones válidas (estado inicial: ACTIVO):
ACTIVO → INACTIVO | BLOQUEADO
INACTIVO → ACTIVO
BLOQUEADO → ACTIVO
Excepciones:
mismo estado → PersonaConMismoEstadoException
transición no válida → CambioEstadoPersonaIllegalException
BLOQUEADO sin justificación → FaltaJustificacionException
nombre desconocido en EstadosDonante → TipoDesconocidoException
PersonaJuridica
private razonSocial: String
private tipo: TipoPersonaJuridica
private rubro: String
private representantes: List<Representante>
public agregarRepresentante(representante: Representante): void
Representante
private id: UUID
private nombre: String
private apellido: String
private email: String
private contactos: List<MedioDeContacto>
0..*
0..*
«enum»
TipoPersonaJuridica
GUBERNAMENTAL
ONG
EMPRESA
INSTITUCION
0..1
contacto
Email
private normalizarValor(): void «@PrePersist, @PreUpdate»
Telefono
Whatsapp
0..*
contactos
0..1
medioContactoPredeterminado
0..1
0..*
«abstracta»
MedioDeContacto
protected id: UUID
protected valor: String
entidad
EntidadBeneficiaria
private id: UUID
private razonSocial: String
private direccion: Direccion
private contactos: List<MedioDeContacto>
private representantes: List<Representante>
private campanias: List<Campania>
public agregarCampania(campania: Campania): void
public agregarNecesidadACampania(campania: Campania, necesidad: Necesidad): void
public obtenerEmail(): String
public contarNecesidadesCompatiblesCon(subcategoria: String): int
0..1
«embeddable»
Direccion
private calle: String
private numero: int
private localidad: Localidad
private codigoPostal: String
0..1
«embeddable»
Localidad
private nombre: String
private provincia: Provincia
0..1
«embeddable»
Provincia
private nombre: String
0..1
necesidad
Campania
private idCampania: UUID
private idEntidad: UUID {readOnly}
private fechaInicio: LocalDate
private fechaFin: LocalDate
private descripcionGeneral: String
private necesidades: List<Necesidad>
public agregarNecesidad(necesidad: Necesidad): void
public buscarNecesidad(necesidadId: UUID): Optional<Necesidad>
public eliminarNecesidad(necesidadId: UUID): boolean
public necesidadesCompatiblesCon(subcategoria: String): List<Necesidad>
0..*
0..*
«abstracta»
Necesidad
protected id: UUID
protected nombre: String
protected descripcion: String
protected fechaRegistro: LocalDate
protected cantidadObjetivo: int
protected cantidadRecibida: int
public recibirDonacion(cantidad: int): void
public esCompatibleCon(subcategoria: String): boolean
private contieneTexto(texto: String, objetivo: String): boolean
public estaSatisfecha(): boolean
NecesidadExtraordinaria
public estaSatisfecha(): boolean
NecesidadRecurrente
private periodo: Periodicidad
private fechaInicioPeriodo: LocalDate
public periodoVencido(referencia: LocalDate): boolean
public obtenerOGenerarPeriodoActual(hoy: LocalDate): boolean
public estaSatisfecha(): boolean
periodicidades
«enum»
Periodicidad
SEMANAL(7)
MENSUAL(30)
CUATRIMESTRAL(120)
ANUAL(365)
private dias: int
public getDias(): int
0..1
Interfaz de Spring Data (Entrega 5).
El id se asigna antes de guardar: sin isNew()
Spring Data haría un SELECT previo a cada INSERT.
nueva = true hasta @PostPersist / @PostLoad.
«interface»
Persistable
UUID
public getId(): UUID
public isNew(): boolean